GROWURK

Web Development Internship Training Program

WEEK 7 — PRACTICAL LABS

Next.js 14 · App Router · TypeScript · Server Actions · Turborepo

5 Hands-On Labs · Project 4: Advanced Architecture Refactor

Lab	Title
Lab 1	Migrate a Next.js App to the App Router
Lab 2	Build with React Server Components & Client Components
Lab 3	Add TypeScript: Interfaces, Generics & Utility Types
Lab 4	Replace API Routes with Server Actions
Lab 5	Structure a Codebase with Feature Folders & Turborepo

LAB 01

Migrate a Next.js App to the App Router

⏱ 90 minutes ◆ Intermediate

OBJECTIVE

Convert an existing Next.js application from the Pages Router to the App Router. You will recreate routes using the new file conventions, implement nested layouts, and set up loading and error states.

REAL-WORLD SCENARIO

Your team has inherited a Next.js 12 codebase built with the Pages Router. The tech lead has decided to migrate to Next.js 14 App Router to unlock React Server Components and streaming. You have been assigned the first milestone: migrate the routing structure without breaking any existing functionality.

PREREQUISITES

- An existing Next.js project (or create a fresh one: npx create-next-app@latest --use-npm)
- Node.js 18.17.0 or later
- Basic familiarity with Next.js Pages Router (from Weeks 1–6)

LAB INSTRUCTIONS

Part 1 — Create the App Router Structure

1. Create a new Next.js 14 project with App Router enabled (it is the default):

```
npx create-next-app@latest week7-app --typescript --tailwind --app
cd week7-app
```

2. Examine the generated app/ directory structure — notice there is no pages/ directory:

```
app/
├── layout.tsx       ← Root layout (replaces _app.tsx)
├── page.tsx         ← Home route  /
├── globals.css
└── favicon.ico
```

3. Create the following route structure by adding folders and files:

```
app/
├── layout.tsx              ← Root layout
├── page.tsx                ← /
├── (marketing)/            ← Route group — no URL segment
│   ├── about/page.tsx      ← /about
│   └── pricing/page.tsx    ← /pricing
├── (shop)/                 ← Route group
│   ├── layout.tsx          ← Layout only for shop routes
│   ├── products/
│   │   ├── page.tsx        ← /products
│   │   └── [id]/page.tsx   ← /products/[id]  (dynamic)
```

```
│   └── cart/page.tsx       ← /cart
└── dashboard/
    ├── layout.tsx          ← Persistent dashboard sidebar
    ├── page.tsx            ← /dashboard
    └── orders/page.tsx     ← /dashboard/orders
```

Part 2 — Build the Root Layout

1. Open app/layout.tsx and build a root layout with a persistent navigation bar:

```
import type { Metadata } from 'next';
import { Inter } from 'next/font/google';
import './globals.css';

const inter = Inter({ subsets: ['latin'] });

export const metadata: Metadata = {
  title: { template: '%s | My Store', default: 'My Store' },
  description: 'A full-stack e-commerce store',
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang='en'>
      <body className={inter.className}>
        <nav className='h-16 border-b flex items-center px-6 gap-6'>
          <a href='/' className='font-bold text-lg'>My Store</a>
          <a href='/products'>Products</a>
          <a href='/cart'>Cart</a>
        </nav>
        <main className='container mx-auto px-6 py-8'>{children}</main>
      </body>
    </html>
  );
}
```

2. Create the (shop)/layout.tsx with a shop-specific breadcrumb bar — this layout only applies to /products and /cart:

```
export default function ShopLayout({ children }: { children: React.ReactNode }) {
  return (
    <div>
      <div className='text-sm text-gray-500 mb-4'>Shop › Products</div>
      {children}
    </div>
  );
}
```

Part 3 — Loading and Error States

3. Add a loading.tsx to the products route to show a skeleton while data loads:

```
// app/(shop)/products/loading.tsx
export default function Loading() {
  return (
    <div className='grid grid-cols-3 gap-4'>
      {Array.from({ length: 6 }).map((_, i) => (
```

```
        <div key={i} className='h-48 bg-gray-200 rounded-lg animate-pulse' />
      ))}
    </div>
  );
}
```

4. Add an error.tsx to the products route. Error components MUST be Client Components:

```
// app/(shop)/products/error.tsx
'use client'; // Required — error boundaries must be client components

export default function Error({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  return (
    <div className='text-center py-12'>
      <h2 className='text-xl font-semibold mb-4'>Something went wrong</h2>
      <p className='text-gray-600 mb-6'>{error.message}</p>
      <button onClick={reset} className='bg-blue-600 text-white px-4 py-2
rounded'>
        Try Again
      </button>
    </div>
  );
}
```

5. Test the loading state by adding an artificial delay in app/(shop)/products/page.tsx:

```
// Simulates a slow database query
async function getProducts() {
  await new Promise(resolve => setTimeout(resolve, 2000)); // 2 second delay
  return [{ id: 1, name: 'Headphones', price: 79.99 }];
}

export default async function ProductsPage() {
  const products = await getProducts();
  return <div>{products.map(p => <div key={p.id}>{p.name}</div>)}</div>;
}
```

6. Run npm run dev and navigate to /products — observe the loading skeleton, then the products.

EXPECTED OUTPUT

- All routes accessible: /, /about, /pricing, /products, /products/1, /cart, /dashboard, /dashboard/orders
- The root nav bar persists on all pages (never re-renders on navigation)
- The (shop) breadcrumb bar appears on /products and /cart but NOT on /about or /dashboard
- Navigating to /products shows the loading skeleton for 2 seconds, then the product list
- Route groups (marketing) and (shop) do not appear in the URL

TROUBLESHOOTING TIPS

Error: You're importing a component that needs useState A server component is importing a client-only library. Add 'use client' to the file that uses the browser API, or move the browser code into a separate client component.

Layout not persisting Ensure your nav bar is in layout.tsx and NOT in page.tsx. Pages re-render on navigation; layouts do not.

CHALLENGE TASK

Optional Extension: Implement a modal using parallel routes. Create a @modal slot in the (shop) layout and an intercepting route for /products/[id] that renders a product detail modal when navigating from the product list, but a full product page when accessed directly via URL.

LAB 02

React Server Components & the Client Boundary

⏱ 90 minutes ◆ Intermediate

OBJECTIVE

Build a product catalog page that uses React Server Components for data fetching and Client Components only for interactive elements. Practice identifying and placing the server/client boundary optimally.

REAL-WORLD SCENARIO

Your product page currently loads all data client-side with useEffect — users see a flash of empty content on every visit, and the page is slow on mobile. Your tech lead says: 'Move the data fetching to the server. The JavaScript bundle is too big and it shows in our Core Web Vitals.'

PREREQUISITES

- Completion of Lab 1 (App Router structure set up)
- Prisma configured with products in the database (from Week 6 Lab 1)

LAB INSTRUCTIONS

Part 1 — Server Component Data Fetching

7. Create a server component that fetches products directly from the database. No useEffect, no fetch — just async/await:

```
// app/(shop)/products/page.tsx
import { prisma } from '@/lib/prisma';
import { ProductGrid } from './ProductGrid';

// This component runs ONLY on the server
export default async function ProductsPage({
  searchParams,
}: {
  searchParams: { category?: string; q?: string };
}) {
  const products = await prisma.product.findMany({
    where: {
      ...(searchParams.category && { category: { name: searchParams.category } }),
      ...(searchParams.q && { name: { contains: searchParams.q, mode:
'insensitive' } }),
    },
    include: { category: true },
    orderBy: { createdAt: 'desc' },
  });

  return (
    <div>
      <h1 className='text-3xl font-bold mb-8'>Products</h1>
      <ProductGrid products={products} />
```

```
    </div>
  );
}
```

Part 2 — Client Components for Interactivity

8. Create ProductGrid as a client component because it needs the 'Add to Cart' click handler:

```
// app/(shop)/products/ProductGrid.tsx
'use client';

import { useState } from 'react';

type Product = { id: number; name: string; price: number; category: { name: string
} };

export function ProductGrid({ products }: { products: Product[] }) {
  const [cart, setCart] = useState<number[]>([]);

  const addToCart = (id: number) => setCart(prev => [...prev, id]);

  return (
    <div>
      <p className='mb-4 text-sm text-gray-600'>{cart.length} item(s) in cart</p>
      <div className='grid grid-cols-1 md:grid-cols-3 gap-6'>
        {products.map(product => (
          <div key={product.id} className='border rounded-lg p-4'>
            <span className='text-xs text-purple-
600'>{product.category.name}</span>
            <h3 className='font-semibold mt-1'>{product.name}</h3>
            <p className='text-lg font-bold mt-2'>${product.price.toFixed(2)}</p>
            <button
              onClick={() => addToCart(product.id)}
              className='mt-3 w-full bg-purple-600 text-white py-2 rounded'
            >
              Add to Cart
            </button>
          </div>
        ))}
      </div>
    </div>
  );
}
```

Part 3 — Search with useOptimistic

9. Create a SearchBar client component that updates the URL search params for server-side filtering:

```
// app/(shop)/products/SearchBar.tsx
'use client';

import { useRouter, usePathname, useSearchParams } from 'next/navigation';
import { useOptimistic, useTransition } from 'react';

export function SearchBar({ initialQuery }: { initialQuery: string }) {
  const router = useRouter();
```

```
  const pathname = usePathname();
  const searchParams = useSearchParams();
  const [isPending, startTransition] = useTransition();
  const [optimisticQuery, setOptimisticQuery] = useOptimistic(initialQuery);

  const handleSearch = (q: string) => {
    setOptimisticQuery(q); // Instant UI update
    startTransition(() => {
      const params = new URLSearchParams(searchParams);
      q ? params.set('q', q) : params.delete('q');
      router.push(`${pathname}?${params.toString()}`);
    });
  };

  return (
    <div className='relative mb-6'>
      <input
        value={optimisticQuery}
        onChange={e => handleSearch(e.target.value)}
        placeholder='Search products...'
        className='w-full border rounded-lg px-4 py-2 pr-10'
      />
      {isPending && <span className='absolute right-3 top-2 text-gray-
400'>...</span>}
    </div>
  );
}
```

10. Add the SearchBar to the ProductsPage server component above the ProductGrid.

EXPECTED OUTPUT

- The products page loads with NO client-side data fetching (no loading flash)
- Right-clicking the page → View Page Source shows the product HTML already present — proof of SSR
- 'Add to Cart' button updates the cart count instantly
- Typing in the search bar shows an instant optimistic update, then filters server-side
- Network tab: no fetch requests to /api/products — data comes directly from the server render

TROUBLESHOOTING TIPS

Error: prisma is not defined in a client component You are trying to call the database from a client component. Move all Prisma calls up to a server component (no 'use client' directive) and pass the data down as props.

useOptimistic causes hydration errors Ensure the component using useOptimistic has 'use client' at the top. useOptimistic is a React hook — it only works in client components.

CHALLENGE TASK

Optional Extension: Add Suspense streaming to the product page. Wrap the ProductGrid in a <Suspense fallback={<Loading />}> in the server component. This streams the layout to

the browser instantly and streams the products grid when ready, without waiting for the full page to render.

LAB 03

Add TypeScript — Interfaces, Generics & Utility Types

⏱ 75 minutes ◆ Intermediate

OBJECTIVE

Add TypeScript types to your application's data layer, API responses, and form inputs. Practice writing generics and using utility types to avoid code duplication and catch errors before runtime.

REAL-WORLD SCENARIO

Your codebase is growing and bugs are appearing: a developer passed a string where a number was expected, causing a payment calculation to break in production. The team lead says: 'We are adding TypeScript. I need you to type our API responses and Prisma models so this never happens again.'

PREREQUISITES

- TypeScript configured in Next.js (default when using --typescript flag)
- Prisma schema from Lab 1 of Week 6

LAB INSTRUCTIONS

Part 1 — Domain Types

11. Create a types/ folder and define your domain interfaces in types/index.ts:

```
// types/index.ts

export interface Product {
  id: number;
  name: string;
  price: number;
  stock: number;
  category: Category;
  createdAt: Date;
}

export interface Category {
  id: number;
  name: string;
}

export interface Order {
  id: number;
  userId: number;
  total: number;
  status: OrderStatus;
  items: OrderItem[];
  createdAt: Date;
```

```
}

export type OrderStatus = 'PENDING' | 'PAID' | 'SHIPPED' | 'DELIVERED' |
'CANCELLED';

export interface OrderItem {
  id: number;
  productId: number;
  product: Product;
  quantity: number;
  price: number;
}

export interface User {
  id: number;
  email: string;
  name: string | null;
  createdAt: Date;
}
```

Part 2 — Generic API Response Wrapper

12. Create a generic ApiResponse type and a typed fetch helper:

```
// lib/api.ts

export type ApiResponse<T> =
  | { success: true;  data: T }
  | { success: false; error: string };

export async function fetchApi<T>(url: string): Promise<ApiResponse<T>> {
  try {
    const res = await fetch(url);
    if (!res.ok) {
      return { success: false, error: `HTTP ${res.status}: ${res.statusText}` };
    }
    const data = await res.json() as T;
    return { success: true, data };
  } catch (err) {
    return { success: false, error: 'Network error' };
  }
}

// Usage — TypeScript infers the full type:
// const result = await fetchApi<Product[]>('/api/products');
// if (result.success) {
//   result.data[0].name  ← TypeScript knows this is a string
// }
```

Part 3 — Utility Types in Practice

13. Use TypeScript utility types to derive useful sub-types from your domain types:

```
// types/forms.ts
import type { Product, User } from './index';

// CreateProductInput: all Product fields except auto-generated ones
```

```
export type CreateProductInput = Omit<Product, 'id' | 'createdAt' | 'category'> &
{
  categoryId: number;
};

// UpdateProductInput: every field optional except id
export type UpdateProductInput = Partial<Omit<Product, 'id'>> & { id: number };

// PublicUser: User without sensitive fields
export type PublicUser = Omit<User, 'email'>;

// ProductSummary: only the fields needed for a product card
export type ProductSummary = Pick<Product, 'id' | 'name' | 'price'> & {
  categoryName: string;
};

// ProductLookup: a dictionary of products by ID
export type ProductLookup = Record<number, Product>;
```

Part 4 — Type Guards

14. Write a type guard to safely narrow union types from API responses:

```
// lib/guards.ts
import type { ApiResponse } from './api';

// Type guard: narrows ApiResponse<T> to the success branch
export function isSuccess<T>(response: ApiResponse<T>): response is { success:
true; data: T } {
  return response.success === true;
}

// Usage:
// const result = await fetchApi<Product[]>('/api/products');
// if (isSuccess(result)) {
//   console.log(result.data); // TypeScript knows this is Product[]
// } else {
//   console.error(result.error); // TypeScript knows this is string
// }

// Type guard for OrderStatus
const validStatuses = ['PENDING', 'PAID', 'SHIPPED', 'DELIVERED', 'CANCELLED'] as
const;

export function isOrderStatus(value: string): value is typeof
validStatuses[number] {
  return (validStatuses as readonly string[]).includes(value);
}
```

15. Apply the types throughout your application — add type annotations to your Prisma query results, API routes, and component props. Notice how VS Code highlights any type mismatches immediately.

EXPECTED OUTPUT

- Zero TypeScript errors in VS Code for all typed files
- Hovering over a Prisma query result shows the inferred type with all fields

- Attempting to pass a string where number is expected shows a red underline before running
- fetchApi<Product[]> returns a fully typed response — no 'any' types
- UpdateProductInput correctly makes all fields optional except id

TROUBLESHOOTING TIPS

Type 'any' keeps appearing You are likely using JSON.parse() without a type assertion, or a library without TypeScript definitions. Install @types/library-name or add an explicit generic: JSON.parse(text) as MyType.

Prisma types conflict with my interfaces Prefer using Prisma's generated types directly: import type { Product } from '@prisma/client' rather than redefining them. Derive your form types from Prisma types using Omit and Pick.

CHALLENGE TASK

Optional Extension: Create a typed pagination generic: type PaginatedResponse<T> = { data: T[]; total: number; page: number; pageSize: number; totalPages: number }. Then write a generic paginate<T>() server function that accepts any Prisma model query and returns PaginatedResponse<T>.

LAB 04

Replace API Routes with Server Actions

⏱ 75 minutes ◆ Intermediate

OBJECTIVE

Refactor a form submission workflow from a traditional POST /api/route pattern to a Next.js 14 Server Action. Add optimistic updates so the UI feels instant, and handle loading and error states cleanly.

REAL-WORLD SCENARIO

Your product creation form currently makes a fetch() call to POST /api/admin/products. That is 40+ lines across two files: the route handler and the form component. Your tech lead points to a 5-line Server Action that does the same thing and says: 'This is why we upgraded to Next.js 14. Clean it up.'

PREREQUISITES

- Lab 1 App Router structure complete
- Lab 3 TypeScript types defined (CreateProductInput)
- Prisma connected to Railway PostgreSQL

LAB INSTRUCTIONS

Part 1 — Create a Server Action

16. Create app/actions/product.actions.ts. The 'use server' directive at the file level marks ALL exports as server actions:

```
// app/actions/product.actions.ts
'use server';

import { prisma } from '@/lib/prisma';
import { revalidatePath } from 'next/cache';
import { redirect } from 'next/navigation';
import type { CreateProductInput } from '@/types/forms';

export async function createProduct(input: CreateProductInput) {
  // Validate
  if (!input.name || input.price <= 0) {
    return { error: 'Name and a positive price are required.' };
  }

  // Create in database
  const product = await prisma.product.create({
    data: {
      name: input.name,
      price: input.price,
      stock: input.stock,
      categoryId: input.categoryId,
    },
```

```
  });

  // Invalidate the products page cache so it shows the new product
  revalidatePath('/products');

  return { success: true, product };
}

export async function deleteProduct(productId: number) {
  await prisma.product.delete({ where: { id: productId } });
  revalidatePath('/products');
}
```

Part 2 — Build the Form Component

17. Create a client component that calls the server action and handles states:

```
// app/admin/AddProductForm.tsx
'use client';

import { useTransition, useState } from 'react';
import { createProduct } from '@/app/actions/product.actions';

export function AddProductForm({ categoryId }: { categoryId: number }) {
  const [isPending, startTransition] = useTransition();
  const [error, setError] = useState<string | null>(null);
  const [success, setSuccess] = useState(false);

  const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    const form = e.currentTarget;
    const data = new FormData(form);

    startTransition(async () => {
      const result = await createProduct({
        name: data.get('name') as string,
        price: Number(data.get('price')),
        stock: Number(data.get('stock')),
        categoryId,
      });

      if (result.error) {
        setError(result.error);
      } else {
        setSuccess(true);
        form.reset();
      }
    });
  };

  return (
    <form onSubmit={handleSubmit} className='space-y-4 max-w-md'>
      {error && <p className='text-red-600 text-sm'>{error}</p>}
      {success && <p className='text-green-600 text-sm'>Product created!</p>}

      <input name='name' placeholder='Product name' required
        className='w-full border rounded px-3 py-2' />
      <input name='price' type='number' step='0.01' placeholder='Price'
        className='w-full border rounded px-3 py-2' />
```

```
      <input name='stock' type='number' placeholder='Stock'
        className='w-full border rounded px-3 py-2' />

      <button type='submit' disabled={isPending}
        className='bg-purple-600 text-white px-6 py-2 rounded disabled:opacity-
50'>
        {isPending ? 'Creating...' : 'Create Product'}
      </button>
    </form>
  );
}
```

Part 3 — Optimistic Delete with useOptimistic

18. Build a product list with optimistic deletion — the product disappears instantly before server confirms:

```
// app/admin/ProductList.tsx
'use client';

import { useOptimistic, useTransition } from 'react';
import { deleteProduct } from '@/app/actions/product.actions';
import type { Product } from '@prisma/client';

export function ProductList({ products }: { products: Product[] }) {
  const [optimisticProducts, removeOptimistic] = useOptimistic(
    products,
    (state, deletedId: number) => state.filter(p => p.id !== deletedId)
  );
  const [, startTransition] = useTransition();

  const handleDelete = (id: number) => {
    startTransition(async () => {
      removeOptimistic(id); // Instant UI update
      await deleteProduct(id); // Server catches up
    });
  };

  return (
    <ul className='space-y-2'>
      {optimisticProducts.map(product => (
        <li key={product.id} className='flex justify-between items-center border
rounded p-3'>
          <span>{product.name} — ${product.price}</span>
          <button onClick={() => handleDelete(product.id)}
            className='text-red-600 hover:underline text-sm'>
            Delete
          </button>
        </li>
      ))}
    </ul>
  );
}
```

EXPECTED OUTPUT

- Submitting the AddProductForm creates a product in the database with no separate API route file

- The products page at /products updates automatically after creation (revalidatePath works)
- Clicking 'Delete' removes the product from the UI instantly — before the database confirms
- If the delete fails, the product reappears (optimistic rollback)
- The form shows 'Creating...' during the pending state and 'Product created!' on success

TROUBLESHOOTING TIPS

Error: Server Actions cannot be used from a Server Component directly Server Actions called with startTransition must be inside a 'use client' component. The action itself (in product.actions.ts) stays server-only; only the calling component needs 'use client'.

revalidatePath not updating the page Ensure the path matches exactly. If your products route is at /products, use revalidatePath('/products'). Use revalidatePath('/', 'layout') to invalidate all pages.

CHALLENGE TASK

Optional Extension: Add Zod validation to the createProduct server action. Install zod, define a CreateProductSchema with min/max constraints on name length, price range, and stock. Return structured validation errors as an array so the form can display per-field error messages.

LAB 05

Feature Folders, Barrel Exports & Turborepo

⏱ 3 hours ◆ Advanced — Architecture Lab

OBJECTIVE

Reorganize your e-commerce codebase into a feature-based folder structure, implement barrel exports, and set up a basic Turborepo monorepo with a shared types package. This is the architecture used by professional engineering teams.

REAL-WORLD SCENARIO

Your codebase has 40+ files all in a flat components/ directory. A new developer joined the team and spent 2 hours just finding the checkout-related files. The engineering manager calls a meeting: 'We need to reorganize before we add more features. I want to be able to look at the folder structure and immediately understand what the application does.'

LAB INSTRUCTIONS

Part 1 — Feature-Based Folder Refactor

19. Reorganize your Next.js project from type-based to feature-based folders:

```
src/
├── app/                    ← Next.js App Router (routing only)
│   ├── (shop)/
│   │   ├── products/page.tsx   ← Imports from features/products
│   │   └── cart/page.tsx       ← Imports from features/cart
│   └── layout.tsx
│
├── features/               ← All business logic lives here
│   ├── products/
│   │   ├── components/
│   │   │   ├── ProductCard.tsx
│   │   │   ├── ProductGrid.tsx
│   │   │   └── SearchBar.tsx
│   │   ├── actions/
│   │   │   └── product.actions.ts
│   │   ├── hooks/
│   │   │   └── useProductFilter.ts
│   │   ├── types/
│   │   │   └── product.types.ts
│   │   └── index.ts        ← Barrel export
│   │
│   ├── cart/
│   │   ├── components/
│   │   ├── hooks/useCart.ts
│   │   ├── types/cart.types.ts
│   │   └── index.ts
│   │
│   └── auth/
│       ├── components/
│       └── index.ts
│
└── lib/                    ← Framework setup (shared utilities)
```

```
    ├── prisma.ts
    ├── stripe.ts
    └── resend.ts
```

Part 2 — Barrel Exports

20. Create the barrel index.ts for the products feature:

```
// features/products/index.ts

// Re-export everything public from this feature
export { ProductCard } from './components/ProductCard';
export { ProductGrid } from './components/ProductGrid';
export { SearchBar } from './components/SearchBar';
export { createProduct, deleteProduct } from './actions/product.actions';
export { useProductFilter } from './hooks/useProductFilter';
export type { Product, CreateProductInput } from './types/product.types';
```

21. Update imports in your app/ pages to use the barrel:

```
// Before (type-based imports):
import { ProductGrid } from '@/components/ProductGrid';
import { createProduct } from '@/actions/product.actions';
import type { Product } from '@/types';

// After (feature barrel import):
import { ProductGrid, createProduct, type Product } from '@/features/products';
```

Part 3 — Turborepo Monorepo Setup

22. Initialize a new Turborepo workspace (do this in a fresh directory outside your Next.js project):

```
npx create-turbo@latest growurk-platform
cd growurk-platform
```

23. Examine the generated structure:

```
growurk-platform/
├── apps/
│   ├── web/          ← Your Next.js app
│   └── docs/         ← (optional) Documentation site
├── packages/
│   ├── ui/           ← Shared React components
│   ├── eslint-config/ ← Shared ESLint config
│   └── typescript-config/ ← Shared tsconfig
├── package.json
└── turbo.json
```

24. Move your Next.js project into apps/web/ and create a shared types package:

```
# Create packages/types/
mkdir -p packages/types/src

# packages/types/package.json
{
  "name": "@repo/types",
```

```
  "version": "0.0.1",
  "main": "./src/index.ts",
  "types": "./src/index.ts",
  "devDependencies": {
    "typescript": "^5"
  }
}
```

25. Move your domain types (Product, Order, User) into packages/types/src/index.ts
26. Add the package as a dependency in apps/web/package.json and import from @repo/types:

```
// apps/web/package.json
{
  "dependencies": {
    "@repo/types": "*"
  }
}

// In any file in apps/web/:
import type { Product, Order } from '@repo/types';
```

27. Run the build pipeline from the root:

```
pnpm install        # Install all workspace dependencies
pnpm turbo build    # Build all packages in the correct order
```

EXPECTED OUTPUT

- The features/ directory clearly communicates what the application does by folder name alone
- Importing from '@/features/products' gives access to all product components, actions, and types
- The Turborepo build completes successfully with cache hits on unchanged packages
- Changing a type in packages/types triggers a rebuild only in packages that depend on it
- VS Code IntelliSense resolves @repo/types imports correctly with full type information

TROUBLESHOOTING TIPS

Circular import error with barrel exports Barrel exports create circular imports when Feature A's barrel imports from Feature B which imports from Feature A. Keep cross-feature imports direct (not through barrels) and use the lib/ folder for truly shared code.

Turborepo: Cannot find module @repo/types Run pnpm install from the workspace root to link all packages. If using npm workspaces instead of pnpm, use npm install from the root. Never npm install inside individual packages.

PROJECT 4 SUBMISSION CHECKLIST

Requirement	Done When

App Router	All routes migrated from pages/ to app/ with layouts, loading, and error files
Server Components	Data fetching moved from useEffect to async server components
TypeScript	All files have explicit types — zero 'any' types in the codebase
Server Actions	At least 2 form mutations use server actions instead of API routes
Feature Folders	Codebase organized into features/ directory with barrel exports
Deployment	Deployed to Vercel (frontend) with Railway PostgreSQL still connected